

WILEY

INTERNATIONAL
TRANSACTIONS
IN OPERATIONAL
RESEARCHIntl. Trans. in Op. Res. 29 (2022) 2316–2345
DOI: 10.1111/itor.12901

Heuristics for flights arrival scheduling at airports

Mohammad Mahdi Ahmadian  and Amir Salehipour* *School of Mathematical and Physical Sciences, University of Technology, Sydney, Australia**E-mail: mohammadmahdi.ahmadian@student.uts.edu.au [Ahmadian]; amir.salehipour@uts.edu.au [Salehipour]*

Received 7 November 2019; received in revised form 3 October 2020; accepted 23 October 2020

Abstract

We develop an efficient matheuristic algorithm for the aircraft landing problem (ALP). The ALP aims to schedule aircraft landings such that the total deviation from target arrival times is minimized. We propose a relax-and-solve (R&S) algorithm that operates by performing a set of “relax” and “solve” iterations. The relax procedure destructs a sequence of aircraft landings, and the solve procedure re-constructs a complete sequence and schedules the aircraft landings. We compare the proposed algorithm and the state-of-the-art algorithm for the ALP and also the solver CPLEX, and show that our algorithm obtains all best-known solutions within one minute, even for instances including 500 aircraft. Those characteristics of the algorithm are very important for practical settings. In particular, the typical short time window available for planning the aircraft landings at busy airports demands quick delivery of quality landing schedules (or updating the current schedule), and fast and effective algorithms are therefore paramount.

Keywords: aircraft landing; weighted earliness and tardiness minimization; heuristic; matheuristic; relaxation neighborhood; relax-and-solve

1. Introduction

It is widely believed that a significant amount of delays experienced at airports are often generated by an inefficient management of runway capacity. Therefore, many studies identify airports as the bottlenecks of the air transportation system (Furini et al., 2015). The runway scheduling problem (RSP) aims to improve runway utilization by allocating the aircraft to runways and optimally determining the landing and take-off time of the aircraft so that a performance criterion is optimized, for example, minimizing the total delays in landing and take-off operations. The RSP is known to have significant impact on other operations occurring at airports, for example, changes to the gate assignments (Zhang and Klabjan, 2017). The RSP has been referred to as the aircraft sequencing problem (ASP) and aircraft landing problem (ALP), though it seems there is no exact definition for ASP and ALP. For example, Furini et al. (2015) define ASP the problem of optimally assigning an

*Corresponding author.

© 2020 The Authors.

International Transactions in Operational Research © 2020 International Federation of Operational Research Societies

Published by John Wiley & Sons Ltd, 9600 Garsington Road, Oxford OX4 2DQ, UK and 350 Main St, Malden, MA02148, USA.

airport's runways to the arrival and departure operations (inbound and outbound traffic), as well as optimally scheduling those operations, whereas ALP is known to be dealing with only scheduling the arrival operations (inbound traffic; Beasley et al., 2000). Following this, the ASP focuses on minimizing the delays in landing and take-off operations and does not penalize early landing and take-off operations, because it is unlikely that aircraft can depart before their estimated departure time. The ASP model of Samá et al. (2017), which they call the aircraft scheduling problem, schedules the arrival and departure operations and takes into account the landing and take-off path in the terminal control area. The ALP's objective, however, considers both earliness and delay in the operations since aircraft can land before their estimated time of arrival. After all, for safety restrictions, the inbound traffic has priority over the outbound and must therefore be managed more importantly and quickly than the outbound traffic.

The objective of the ALP is to obtain a sequence of aircraft landings together with their scheduled landing time such that early and late landings are minimized, and operational constraints are respected. The major operational constraints aim to maintain the minimum separation between every ordered pair of aircraft. The two most applied separation requirements are “radar separation,” which is a lengthwise spacing of five nautical miles (some busy airports may consider less than this) and a vertical spacing of 1000 feet, and “wake turbulence separation (wake vortex or separation time),” which is a time spacing and depends on the type of aircraft. Before operation of Airbus A380, the international civil aviation organization considered three categories of light (L), medium (M), and heavy (H) for wake turbulence. Airbus A380 added the fourth category of super (S). As discussed by Furini et al. (2015), other factors such as aircraft routes and weather conditions may impact separation between aircraft. Hence, the separation times may vary slightly among airports.

A number of different objectives and constraints may be considered for the ALP. One major additional operational constraint is the constrained position shifting (CPS), where each aircraft in the sequence may only deviate by a certain number of positions from its position in the first-come-first-serve (FCFS) order (in which aircraft are sequenced with respect to their arrival times to the airport). The CPS is used to model the “fairness policy” among airlines (Balakrishnan and Chandran, 2010). Some recent studies schedule the landing and take-off operations by considering the landing and take-off path in the terminal control area (see D'Ariano et al., 2015; Samá et al., 2017, 2018). Aircraft must follow certain paths in the controlled airspace surrounding an airport in which they are guided by air traffic controllers. Those paths lead to additional operational constraints. The airport-dependent operational settings may also introduce additional constraints. Two important such constraints include temporary restrictions on utilizing some runways, for example, due to wind direction, weather or congestion or simply due to airline's preference, and nonavailability of some runways for certain arrivals. An example of the latter is Sydney Kingsford Smith airport in which only two of its three runways can accept Airbus A380.

It is a well-known practice that the air traffic controllers order the aircraft by using the FCFS rule, that is, according to their arrival times to the airport. However, a considerable room for improvement is possible with the use of optimization techniques, mainly due to re-ordering the aircraft landings. This has been a motivation for development of various solution approaches for the ALP, ranging from exact methods to heuristics. For example, Bianco et al. (1999) modeled the ALP on one runway as the single machine scheduling problem with ready times and sequence-dependent setup times and the objective function of minimizing the completion times. They developed a

dynamic programming algorithm. Ernst et al. (1999) proposed an exact simplex-based algorithm and a heuristic for the ALP, and obtained optimal solutions for instances with up to 44 aircraft. Beasley et al. (2000) presented a mixed-integer program (MIP) and solved instances with up to 50 aircraft to optimality. A branch-and-price algorithm was later proposed by Wen et al. (2005). They showed that the bounds, which they obtained by applying a column generation algorithm, are stronger than those obtained by the linear programming relaxation (of the MIP). Balakrishnan and Chandran (2010) proposed a dynamic programming algorithm for solving small instances of the ALP with the inclusion of CPS. By using a time discretization approach, Faye (2015) proposed exact and heuristic algorithms. His approach is based on an approximation of the separation times and discretization of the planning horizon. For many small and large instances, his proposed methods deliver tighter lower bounds than those of Beasley et al. (2000). Ghoniem and Farhadi (2015) reformulated the ALP as a set partitioning problem and proposed a column generation algorithm. To solve the pricing problem, they used the solver CPLEX. Later, they proposed a branch-and-price algorithm to address the shortcomings in their earlier study. The pricing problem was now solved as the shortest path problem with time windows and nontriangular separation times (Ghoniem et al., 2015). In spite of availability of exact algorithms for the ALP, practical and large instances still pose a computational challenge. Therefore, heuristics and meta-heuristics have also been proposed to solve the ALP.

The first efforts to use meta-heuristics can be traced back to the work of Abela et al. (1993), in which the genetic algorithm (GA) was used to solve instances with up to 20 aircraft. Hansen (2004) also used the GA for a simplified variant of the problem. Their test problems did not exceed 20 aircraft either. Pinol and Beasley (2006) adapted scatter search and bionomic algorithms. By testing their algorithms on instances as large as 500 aircraft, they obtained optimal solution for instances with up to 50 aircraft. Later, Yu et al. (2011) proposed a two-step algorithm, in which a feasible landing sequence is obtained in the first step and the landing times are determined by a local search procedure in the second step. Their investigation is limited to only one runway, for which superior solutions to those of Pinol and Beasley (2006) were reported. Considering the same instances, Salehipour et al. (2013), Vadamani and Hosseini (2014), Sabar and Kendall (2015), and Girish (2016) have adapted simulated annealing and variable neighborhood search, adaptive large neighborhood search, iterated local search and particle swarm optimization, and obtained optimal solution for small instances, and good quality solutions for large instances. Nevertheless, only a few of those methods are able to obtain the best-known solutions for large instances, due to the computational complexity of the ALP. In addition, because those algorithms include randomized components, their performance may fluctuate and often deteriorates as size of the instances increases. Salehipour and Ahmadian (2017) and Salehipour et al. (2018) developed several algorithms including the variable neighborhood descent (VND) with novel relaxation neighborhoods and iterative greedy algorithms, and attempted to overcome several of the existing limitations. They reported promising solutions for the single runway case, though they did not consider multiple runways. Certain special cases of the ALP have also been studied by Hansen (2004), Salehipour et al. (2009), and Ng et al. (2017), who investigated the arrivals and departures under uncertainty. We refer the interested reader to Bennell et al. (2011) for a comprehensive review of solution techniques for the ALP and to Ng et al. (2018) for an extensive overview and classification of meta-heuristic approaches for airside operations research, which provides details and research directions on this topic, as well as on related problems.

In practice, the schedule of the flight arrivals (i.e., a feasible landing time of aircraft) typically covers a planning horizon of about one hour and is updated every few minutes. This is to ensure that the existing schedule is adjusted so that it accommodates new flights entering into the airport control area. About two to three minutes prior to landing, the schedule is frozen meaning that further changes to the landing schedule will not be considered because the aircraft is too close to landing (Bennell et al., 2011). Following this, generating the landing schedules in a short amount of time is necessary. In addition, the quality of the delivered schedule, which is evaluated against a performance criterion, for example, the total amount of deviations about the target arrival times, may not be overlooked due to its significant service and operational costs. Salehipour (2020) showed that considerable improvements in the schedule and also in service and operational costs are possible with the use of optimization techniques. Nonetheless, the available exact methods are merely suitable in practice because they cannot generate quality schedules quickly. For example, over the tested single runway instances (to be discussed in Section 4), the optimal schedule is only known for instances with up to 50 aircraft. The existing heuristic and meta-heuristics methods, on the other hand, are usually faster than the exact methods, but may not deliver quality schedules. We are only aware of one meta-heuristic that is able to outperform all existing nonexact methods (Girish, 2016). That method includes a combination of hybrid algorithms and different neighborhood structures, meaning that its implementation requires advanced algorithmic techniques and parameters tuning. In this paper, we contribute to solving the ALP by developing a simple and efficient matheuristic algorithm, namely the “relax-and-solve” (R&S) that includes neighborhoods and local search components. The neighborhoods break the original instance of the problem into several smaller instances, through “relaxing” a landing sequence (a set of ordered aircraft arrivals). The local search inside our matheuristic uses an exact solver and aims at “solving” the smaller instances through constructing a sequence and generating a landing schedule. We show that the proposed R&S is able to solve large instances of the ALP in short times, and is therefore ideal for practical settings.

1.1. Paper contributions

In this study, we consider the static ALP on single and multiple runways. The aim is to schedule a set of aircraft for landing in a given planning horizon, and no change to this set (i.e., removal or addition of aircraft or runways) is permitted during the planning. As the input data, we are given minimum separation time units between every pair of aircraft, target landing times of aircraft, as well as their earliest and latest landing times and the penalties per unit of earliness and lateness. The static version has received more attention in the literature than the dynamic variant. In the dynamic version, the landing schedule is regularly updated due to arrival of new aircraft into the system. The research on the dynamic variant has also been pursued. Examples include studies of Beasley et al. (2004), Moser and Hendtlass (2007), and Bennell et al. (2017).

We propose an efficient algorithm for the ALP that is capable of delivering high quality landing schedules. Unlike most studies in the extant literature, our algorithm is a matheuristic method and operates by breaking down the original instance of the problem into smaller instances at the master level, and independently solving the smaller instances at the slave level through optimizing a compact formulation by an exact solver. From a practical point of view, one may acknowledge the conceptual simplicity of our algorithm as one of its merits. Although the state-of-the-art

methods utilize various components within the heuristics and meta-heuristics, leading therefore to implementation of advanced algorithmic techniques and parameters tuning, the structure of our R&S matheuristic is both simple and straightforward. Also, by using an exact solver inside our R&S as the local search, not only do we contribute to a simpler framework, we also commit to using advanced and established solvers. Harnessing the power of solvers for optimizing the smaller instances, which has been mostly overlooked in the literature, is another contribution of the present study.

The contributions of our study can be summarized as follows: (1) we propose an efficient algorithm for the ALP, which operates through iterative applications of destruction and construction, and besides its inherent simplicity is able to produce quality landing schedules for large instances of the ALP, and in short computation times; (2) we propose relaxation neighborhoods for the destruction (relax) procedure, and develop a relaxed formulation for the solve procedure; (3) we develop speed-up techniques aiming at further improving the efficiency of the algorithm; (4) we show that the proposed algorithm, although a matheuristic, is able to obtain optimal solution for all instances with up to 50 aircraft and delivers the best-known solution for larger instances with up to 500 aircraft; and (5) we show that the presented algorithm solves the largest instance in about one minute, and is therefore suitable for practical settings.

1.2. Paper organization

The remainder of this paper is organized as follows. Section 2 explains the ALP, defines the mathematical notations, and formulates the problem as an MIP. Section 3 discusses the proposed R&S matheuristic algorithm. The components of the algorithm, including initial solution generation and improvement procedures, will also be discussed in this section. In addition, a relaxed MIP is developed, which together with the presented speed-up procedures significantly improve the efficiency of solving the problem. The computational outcomes will be discussed in Section 4. Finally, the paper ends with a few conclusions.

2. Problem statement

The ALP is to determine an optimal allocation of a set of aircraft $I = \{1, \dots, n\}$ to land on an airport's runways (a landing sequence), and also an optimal schedule of landings, simultaneously. Although the target landing time of aircraft $i \in I$, that is, T_i is given *a priori*, the scheduled or real landing time x_i , also known as scheduled time of arrival (STA), is an operational-dependent variable that must be decided upon. This implies that an aircraft may not land on its target landing time, or even close to this time. We assume that all runways are identical and accept all types of aircraft.

Minimizing the total cost of early and delayed landings about the given target landing times is the objective of the ALP. If aircraft i lands earlier than its target landing time T_i , that is, $x_i \leq T_i$, its landing is penalized proportionally to the amount of earliness, which is $\alpha_i = \max(0, T_i - x_i)$. On the other hand, if aircraft i is scheduled to land later than its target landing time, that is, $x_i \geq T_i$, the penalty of its late landing is proportional to the amount of its delay, which is $\beta_i = \max(0, x_i - T_i)$. Per unit cost of early and late landing of aircraft i is given by parameters $c_i^- \geq 0$ and $c_i^+ \geq 0$. The

Table 1
The mathematical notations

Sets	
I	Set of aircraft, $I = \{1, \dots, n\}$, $ I = n$, indexed by i .
R	Set of runways, $R = \{1, \dots, m\}$, $ R = m$, indexed by m .
Parameters	
s_{ij}	Separation time units between two ordered aircraft i and j landing on the same runway, $s_{ij} > 0$, $i, j \in I$, $i \neq j$.
T_i	Target landing time of aircraft i , $T_i \geq 0$, $i \in I$.
E_i	Earliest landing time of aircraft i , $E_i \geq 0$, $i \in I$.
L_i	Latest landing time of aircraft i , $L_i \geq E_i$, $i \in I$.
c_i^-	Cost of early landing of aircraft i , $c_i^- \geq 0$, $i \in I$.
c_i^+	Cost of late landing of aircraft i , $c_i^+ \geq 0$, $i \in I$.
Variables	
x_i	Scheduled landing time (STA) of aircraft i , $x_i \geq 0$, $i \in I$.
α_i	Amount of landing earliness of aircraft i (landing before target landing time), $\alpha_i = \max(0, T_i - x_i)$, $\alpha_i \geq 0$.
β_i	Amount of landing lateness of aircraft i (landing after target landing time), $\beta_i = \max(0, x_i - T_i)$, $\beta_i \geq 0$.
y_{ij}	Whether aircraft i lands before aircraft j , $y_{ij} \in \{0, 1\}$, $i, j \in I$, $i \neq j$.
δ_{ij}	Whether aircraft i and j land on the same runway, $\delta_{ij} \in \{0, 1\}$, $i, j \in I$, $i \neq j$.
γ_{ir}	Whether aircraft i is allocated to runway r , $\gamma_{ir} \in \{0, 1\}$, $i \in I$, $r \in R$.

total cost of early and late landings is therefore equal to $\sum_{i=1}^n (c_i^- \alpha_i + c_i^+ \beta_i)$. Considering different per unit cost for early and late landings generalizes the modeling because these two costs may not be identical in practice. Also, in practice, the target landing time of aircraft i may be chosen from the time window $[E_i, L_i]$, which defines the earliest and latest landing times. Therefore, a feasible aircraft landing must be scheduled in this time window, meaning that $E_i \leq x_i \leq L_i$. It follows that $\alpha_i > 0$ if the decision variable x_i lies within the range $[E_i, T_i]$, and $\beta_i > 0$ if it lies within the range $[T_i, L_i]$.

A safe landing requires a separation time $s_{ij} \in \mathbb{R}^+$, $i, j \in I$, $i \neq j$ between every pair of ordered aircraft i and j landing on the same runway. Several factors including the type of aircraft impact the separation times; typically, the choice of runway does not impact the separation times. Also, the separation time between every pair of ordered aircraft landing on the same runway is different than that of landing on different runways. We assume that the separation time between two aircraft landing on different runways is zero time unit. This assumption has been made by others in the literature, for example, see Pinol and Beasley (2006).

The ALP can be formulated as an MIP. Problem P1 shows the mathematical model for ALP (Pinol and Beasley, 2006; Salehipour et al., 2013). In problem P1, the nonnegative variables $x_i \geq 0$, $\forall i \in I$ represent the scheduled landing time of aircraft i . Also, the nonnegative variables $\alpha_i, \beta_i \geq 0$, $\forall i \in I$ show the amount of earliness and tardiness of aircraft i . The binary variables y_{ij} , $\forall i, j \in I$, $i \neq j$ take the value 1 if aircraft i lands before j and take the value 0 otherwise. The binary variables δ_{ij} , $\forall i, j \in I$, $i \neq j$ are introduced to model if aircraft i and j land on the same runway, for which the variables take the value 1, and 0 otherwise. The binary variables γ_{ir} , $\forall i \in I$, $r \in R$ take the value 1 if aircraft i lands on runway r and 0 otherwise. In problem P1, M represents a very large constant, the value of which may be obtained per instance as discussed in Beasley et al. (2000). Table 1 summarizes the mathematical notations used in problem P1.

Problem P1

$$\min z = \sum_{i \in I} (c_i^- \alpha_i + c_i^+ \beta_i). \quad (1)$$

Subject to

$$E_i \leq x_i \leq L_i, \quad \forall i \in I, \quad (2)$$

$$x_i - T_i = \alpha_i - \beta_i, \quad \forall i \in I, \quad (3)$$

$$x_j - x_i \geq s_{ij} \delta_{ij} - M y_{ji}, \quad \forall i, j \in I, i \neq j, \quad (4)$$

$$y_{ij} + y_{ji} = 1, \quad \forall i, j \in I, i \neq j, \quad (5)$$

$$\delta_{ij} \geq \gamma_{ir} + \gamma_{jr} - 1, \quad \forall i, j \in I, i \neq j, r \in R, \quad (6)$$

$$\sum_{r \in R} \gamma_{ir} = 1, \quad \forall i \in I, \quad (7)$$

$$y_{ij}, \delta_{ij} \in \{0, 1\}, \quad \forall i, j \in I, i \neq j, \quad (8)$$

$$\gamma_{ir} \in \{0, 1\}, \quad \forall i \in I, r \in R, \quad (9)$$

$$x_i, \alpha_i, \beta_i \geq 0, \quad \forall i \in I. \quad (10)$$

The objective function (Equation (1)) minimizes the total cost of landing deviations about the target landing times. Constraints (2) ensure that every aircraft lands in its time window. Constraints (3) link the decision variables x_i and parameters T_i to decision variables α_i and β_i . Constraints (4) ensure that if two aircraft i and j land on the same runway, at least s_{ij} time units should be elapsed before aircraft j could be landed on that runway. Given a set of two aircraft, constraints (5) ensure that one lands before the other. Constraints (6) link the decision variables δ_{ij} and γ_{ir} . Constraints (7) imply that every aircraft lands on only one runway. Constraints (8) and (9) force decision variables y_{ij} , δ_{ij} , and γ_{ir} to take only binary values. Finally, constraints (10) impose nonnegativity for decision variables x_i , α_i , and β_i .

We demonstrate a numerical example of the ALP in Appendix B. Next, we present the R&S matheuristic algorithm for the ALP.

3. The proposed R&S algorithm for the ALP

In this section, we propose an R&S matheuristic algorithm for the ALP. Matheuristic algorithms are made by the inter-operation of heuristics and mathematical programming techniques (Boschetti et al., 2009; Maniezzo et al., 2009), and have been developed for a wide range of optimization problems (Doi et al., 2018; Fuentes et al., 2018; Woo and Kim, 2018; Ahmadian et al., 2020).

Algorithm 1. The relax-and-solve (R&S) matheuristic algorithm for the ALP**Input:** An initial sequence π (an ordered set of aircraft) for landing.**while** the stopping condition is not met **do**

Relax();

Solve();

end**return** The best obtained landing schedule;

Given an initial sequence for aircraft landings, the R&S algorithm, which is also known as fix-and-optimize (Helber and Sahling, 2010), obtains an improved sequence by iteratively destructing (relaxing) a sub-sequence of the current sequence, which includes a subset of consecutive aircraft, and re-constructing a feasible sequence by using optimization techniques. Indeed, the “relax” part selects a subset of aircraft to be able to change their position in the landing sequence, whereby the “solve” part determines a (new) landing order for the aircraft in the subset, and generates a complete landing schedule for all aircraft. Algorithm 1 shows a high level presentation of the R&S algorithm.

We use the solver CPLEX (IBM, 2017) as the local search in the solve part of the proposed R&S algorithm. Our R&S is conceptually simple and we will show in Section 4 that it also produces quality solutions for the ALP. Next, we discuss the solution representation, initial sequence generation, relax and solve operations, and the speed-up techniques.

3.1. Generating an initial sequence

We show a sequence for the ALP by an ordered list of $2 \times n$ elements. The landing positions in the sequence are given by the first row and the runway allocations are represented by the second row. For example, the schedule illustrated in Table B3 of Appendix B with $m = 2$ can be represented as follows:

$$\pi = \begin{pmatrix} 3 & 4 & 5 & 6 & 8 & 7 & 9 & 10 & 1 & 14 & 13 & 2 & 12 & 11 & 15 \\ 1 & 1 & 1 & 2 & 1 & 2 & 1 & 2 & 1 & 2 & 2 & 1 & 1 & 2 & 1 \end{pmatrix}.$$

We generate such an initial sequence of landings (which includes a landing sequence and runway allocations) for the R&S by utilizing the earliest target landing time (ETLT) construction algorithm of Salehipour et al. (2013); see Appendix A. The ETLT generates an initial sequence by sorting aircraft in nondecreasing order of their target landing times (i.e., by following the FCFS dispatching rule), and assigning the runways accordingly. We present the pseudocode of the ETLT algorithm and a numerical example of ETLT in Appendices A and B. The ETLT construction heuristic has the running time of $O(n \log n)$.

3.2. The relax procedure

The relax procedure aims at destructing a sub-sequence of a given sequence π . That subset includes a number of consecutive aircraft in the sequence π . The motivation behind the relax procedure

is as follows. An optimal landing schedule for the given landing sequence π can be generated in polynomial time by using problem P1 because given π , problem P1 turns into a linear program. It is clear that such a schedule is optimal for π , and it may not be the global optimal schedule for the problem, implying that if the landing sequence changes, an improved schedule may be produced. Given π , the relax procedure iteratively destructs the landing order of only a small number of aircraft so that the solve procedure is able to (optimally) re-order those aircraft in a short amount of time.

Almost every heuristic and meta-heuristic algorithm for the ALP operates by iteratively changing the landing order of either one aircraft or a small number of aircraft in order to generate new landing sequences. The quality of a generated sequence is evaluated by its schedule. For example, Awasthi et al. (2013) and Girish (2016) proposed polynomial time algorithms for obtaining the optimal schedule, and Furini et al. (2015) and Pinol and Beasley (2006) used the solvers CPLEX and Gurobi for generating the optimal schedule. One difference between our study and the previous ones is that we use the available solvers for both re-sequencing and scheduling.

We may categorize two approaches for the re-sequencing: “complete” and “partial.” In what we call the complete approach, changes are made to either the entire or a part of the sequence. However, the sequence is considered as a whole and the re-sequencing takes place on the complete array of aircraft. For example, Pinol and Beasley (2006) manipulate a sequence through recombination operators and Salehipour et al. (2013), Awasthi et al. (2013), and Sabar and Kendall (2015) perform swap and insertion moves for the same purpose. One major shortcoming regarding that approach is that if an exact solver is used within these algorithms, its role is relegated to only scheduling (i.e., the solver is only used to obtain the optimal landing schedule for a given sequence). Therefore, the solvers are not utilized to change the sequences and obtain improved ones. Indeed, moves made by the solvers are often very fruitful, and contrary to the traditional manipulation techniques (e.g., swap or insertion), which are mostly performed randomly or myopically and without considering their subsequent impact on the rest of the sequence, the solvers can deliver high-quality sequences and schedules. Yet, long sequences associated with large instances are a limiting factor for the available solvers.

Unlike the complete approach, in the partial approach that is suitable for long aircraft sequences, the sequence is broken down into a number of sub-sequences and changes are made to one sub-sequence at a time. The rest of the sequence remains unchanged. Hu and Chen (2005) and Zhan et al. (2009) decomposed the problem into smaller sub-sequences by using the receding horizon control method. To this end, the aircraft whose target times are within a specific receding horizon are selected first, and then are scheduled. We note that the partial approach also allows the exact solvers to be used for re-sequencing. For example, Meng et al. (2011) proposed a sliding window algorithm for the ALP. At each iteration, by sliding the window the algorithm chooses a constant number of unscheduled aircraft, and solves the original ALP associated with those aircraft (i.e., obtains both landing sequence and schedule). In order to maintain the connection to the preceding and succeeding aircraft, the landing times of the previously scheduled aircraft are fixed. The algorithm iterates until all aircraft have their landing times fixed. Clearly, fixing landing time of the aircraft can potentially lead to sub-optimal schedules. A similar idea was investigated by Girish (2016). The study utilized a state-of-the-art algorithm to deliver an optimal landing schedule for a given sequence.

Similar decomposition ideas have also been studied for solving the ASP. To model a variant of the ASP in which each aircraft can be shifted by at most certain positions in the sequence both backward and forward, Furini et al. (2012, 2015) developed an MIP, and proposed a rolling horizon algorithm, which iteratively solves the MIP for a variable time window. The time window rolls forward at each iteration. The last aircraft of the current iteration represents the initial condition for the next iteration. Salehipour and Ahmadian (2017) and Salehipour et al. (2018) adapted the idea of rolling horizon for the ALP, and proposed novel relaxation neighborhoods in which re-sequencing and scheduling are simultaneously performed by the exact solvers. The R&S of the present study shares certain similarities with the rolling horizon framework and those relaxation neighborhoods.

The relax procedure in the proposed R&S breaks down the given landing sequence π into d sub-sequences. We use two parameters of “relaxation center,” denoted by $RC \in \mathbb{Z}^+$, and “relaxation radius,” shown by $RR \in \mathbb{Z}^+$ to form the sub-sequences. The parameter RC determines the center position of the sub-sequence, that is, the aircraft positioned in the middle of the sub-sequence, and the parameter RR determines the size of the sub-sequence. Therefore, the sub-sequence $\pi' \subset \pi$ is formed by the middle position RC and RR positions backward and forward. We call π' the “relaxed” sub-sequence and the remaining sub-sequence(s) the “nonrelaxed.” We set RC and RR such that a certain degree of overlapping between the currently and lately relaxed aircraft is obtained. Ideally, RR should take a small value so that the relaxed sub-sequence involves a small number of aircraft ensuring that the re-sequencing is efficiently performed. The parameter RC is updated during the progress of the R&S algorithm. As such, each iteration of the relax procedure involves formation of a new sub-sequence π' .

Every sub-sequence π' is relaxed through relaxing the precedence constraints of the aircraft in the sub-sequence, letting therefore the aircraft be able to change their landing position in the sequence. The relax procedure does not let the landing position of the aircraft other than those in the current π' be changed. To have a better grasp of the relax procedure, consider the landing sequence illustrated in Table B3 that includes 15 aircraft and one runway (the associated instance and its data are given in Appendix B):

$$\pi = \begin{pmatrix} 3 & 4 & 5 & 6 & 8 & 7 & \mathbf{9} & \mathbf{10} & \mathbf{1} & 14 & 13 & 2 & 12 & 11 & 15 \\ 1 & 1 & 1 & 1 & 1 & 1 & \mathbf{1} & \mathbf{1} & \mathbf{1} & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}.$$

Because the operation of the relax procedure is slightly more complex when multiple runways exist, we first discuss the single runway case. Given $RC = 8$, that is, the eighth position in the sequence and $RR = 1$, the relaxed sub-sequence is $\pi' = \begin{pmatrix} 9, 10, 1 \\ 1, 1, 1 \end{pmatrix}$, which is highlighted in π . Therefore, the relax procedure relaxes the precedence constraints for those aircraft. Figure 1 illustrates how this is performed in connection to the whole sequence. The relaxed aircraft are represented by green and yellow vertices (the green vertex shows the relaxation center). As shown in the figure, arcs originating from the predecessor and ending in the successor of the relaxed sub-sequence, that is, aircraft 7 and 14, vertices of which are shown in red, (i) force the relaxed aircraft to be re-sequenced only within the relaxed sub-sequence, that, between aircraft 7 and 14, and (ii) ensure that the relaxed sub-sequence is not disconnected from the complete sequence, by including the boundary constraints (arcs originating from vertex 7 and ending in vertex 14).

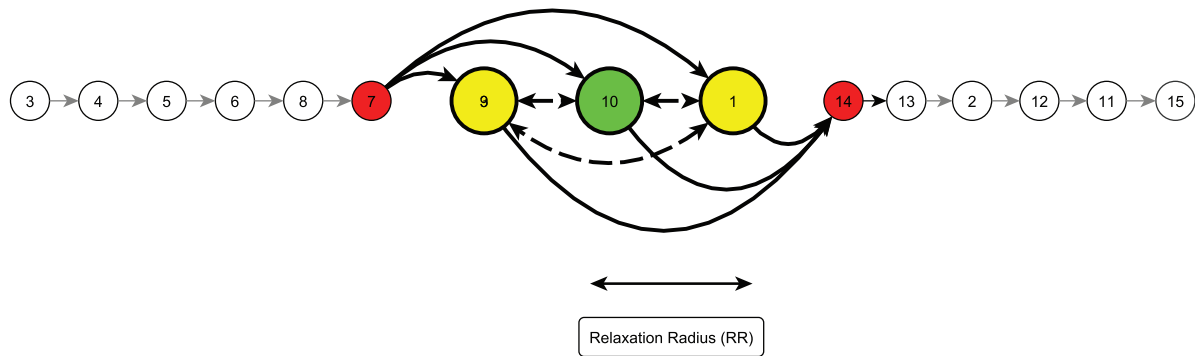


Fig. 1. Operation of the relax procedure in the instance with 15 aircraft and one runway. The conjunctive arcs specify the aircraft that are subject to only scheduling (their sequence is kept as is) and the disjunctive arcs (shown by dashed lines) represent the relaxed aircraft that are subject to both sequencing and scheduling. Arcs from vertex 7 and arcs to vertex 14 ensure that the relaxed aircraft will be re-sequenced only within the relaxed sub-sequence and that they are connected to the whole sequence.

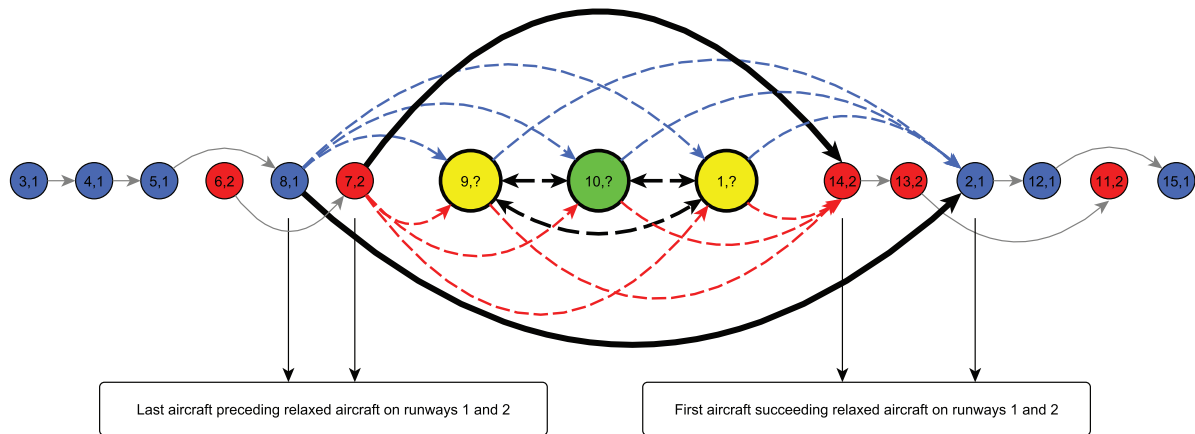


Fig. 2. Operation of the relax procedure in an instance with multiple runways, where the relax sub-sequence includes aircraft 9, 10, and 1 (a pair (i, r) represents aircraft i landing on runway r). The conjunctive arcs specify aircraft that keep their sequence and are subject to only scheduling. Disjunctive arcs within the relaxed sub-sequence (shown by dashed lines) represent the relaxed aircraft, which are subject to runway allocation, re-sequencing and scheduling.

The operation of the relax procedure on multiple runways is more complicated because the relaxed aircraft have to be allocated to runways before any sequencing and scheduling can be performed. Consider the landing sequence illustrated in Table B3 of Appendix B with two runways:

$$\pi = \begin{pmatrix} 3 & 4 & 5 & 6 & 8 & 7 & \mathbf{9} & \mathbf{10} & \mathbf{1} & 14 & 13 & 2 & 12 & 11 & 15 \\ 1 & 1 & 1 & 2 & 1 & 2 & \mathbf{1} & \mathbf{2} & \mathbf{1} & 2 & 2 & 1 & 1 & 2 & 1 \end{pmatrix}.$$

Let the relax sub-sequence include aircraft 9, 10, and 1 (highlighted in the first row). As Fig. 2 shows, aircraft 8 and 7 landing on runways 1 and 2 immediately precede the relaxed sub-sequence. Also, aircraft 14 and 2 that land on runways 2 and 1 immediately succeed the relaxed sub-sequence.

Therefore, the relax procedure operates such that the relaxed sub-sequence will be re-sequenced between those predecessors and successors. The relaxed aircraft can be allocated to each of available runways.

Similar to the single-runway case, the order of aircraft in the nonrelaxed sub-sequence(s) remains unchanged and only the aircraft in the relaxed sub-sequence are subject to runway allocation and re-sequencing. Therefore, depending on the location of the relaxed sub-sequence (the beginning, middle, or end of the sequence) a set of additional precedence constraints are needed to ensure the “connectivity,” that is, the predecessor of the relaxed sub-sequence is also the predecessor to the successor of the relaxed sub-sequence. In our example, those constraints ensure that aircraft 8 lands prior to aircraft 2 on runway 1 (also 7 before 14 on runway 2). Additionally, those constraints impose the connectivity between the nonrelaxed aircraft, that is, aircraft 8 and 2 on runway 1 (aircraft 7 and 14 on runway 2) because if no relaxed aircraft is allocated to runway 1 (and 2), they still hold. In Fig. 2, the highlighted black arcs between vertices 8 and 2 (and 7 and 14) illustrate these constraints (e.g., $x_2 \geq x_8 + s_{82}$ and $x_{14} \geq x_7 + s_{7,14}$). Moreover, the figure includes a set of conjunctive dashed arcs between the nonrelaxed aircraft preceding the relaxed aircraft, that is, originating from vertices 8 and 7, and between the relaxed aircraft and the succeeding nonrelaxed aircraft, that is, ending in vertices 14 and 2. Those arcs only exist if the relaxed aircraft land on the same runway of those nonrelaxed aircraft.

The above discussion leads to the following set of connectivity constraints:

$$x_i \geq x_j + s_{ji} - M(2 - \gamma_{ir} - \gamma_{jr}), \quad i = 9, 10, 1, j = 8, 7, r = 1, 2. \quad (11)$$

Similarly, the following constraints are defined between the relaxed aircraft and the succeeding aircraft, that is, aircraft 2 and 14:

$$x_j \geq x_i + s_{ij} - M(2 - \gamma_{ir} - \gamma_{jr}), \quad i = 9, 10, 1, j = 2, 14, r = 1, 2. \quad (12)$$

It is important to note that the relax procedure ensures that the relaxed aircraft will only be positioned within the relaxed sub-sequence and that they are always connected to the whole sequence. That is the major advantage of the relax procedure over the traditional manipulations.

Next, we explain the solve procedure.

3.3. The solve procedure

The solve procedure aims at generating a (optimal) landing sequence for every sub-sequence π' , and an (optimal) schedule for the whole problem by utilizing the solver CPLEX (IBM, 2017). For this reason, we construct a relaxed formulation of problem P1 by relaxing certain precedence constraints. That relaxed formulation, which we denote as problem P2, is computationally less challenging than solving problem P1 due to less number of precedence constraints.

Problem P2

Constraints (4) in problem P1 determine the landing schedule for an aircraft by considering all scheduled aircraft, and ensuring that the separation times are met. Even a medium-sized instance

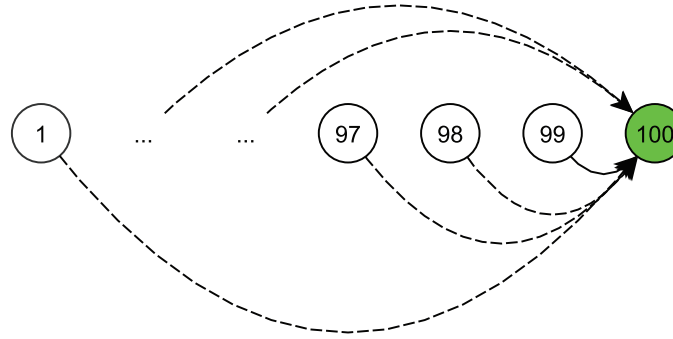


Fig. 3. Only a few immediate precedence constraints might be binding when scheduling an aircraft.

of the ALP may involve a large number of constraints (4). We observe that in the majority of the tested instances such a large number of constraints may be avoided without violating the separation time requirements. Therefore, we obtain problem P2 by relaxing a large number of constraints (4).

Let us start by a given instance with 100 aircraft and one runway. Given a landing sequence, problem P1 determines the landing schedule for every aircraft by considering the landing schedule of all of its preceding aircraft. This is equivalent to inequality (13):

$$x_j \geq \max_{1 \leq i \leq j-1} \{x_i + s_{ij}\}, \quad (13)$$

where aircraft j lands after aircraft i . Consider the landing sequence $\pi = \begin{pmatrix} 1, 2, \dots, 99, 100 \\ 1, 1, \dots, 1, 1 \end{pmatrix}$. Assume that we want to determine the landing schedule for the last aircraft. It follows from constraints (4) in problem P1 that

$$\begin{aligned} x_{100} &\geq x_1 + s_{1,100}, \\ &\vdots \\ x_{100} &\geq x_{98} + s_{98,100}, \\ x_{100} &\geq x_{99} + s_{99,100}. \end{aligned}$$

This is illustrated in Fig. 3. However, because of the characteristic of the separation times, the binding constraints are typically due to certain immediate predecessors, and most likely due to the very immediate predecessor. Also, because problem P2 is built on a given sequence, we only include the smallest number of constraints (4) for the nonrelaxed sub-sequence(s), which is equal to $n - 1$. It should be noted that the number of all constraints (4) in problem P1 is equal to $\sum_{i=1}^{n-1} (i) = \frac{n \times (n-1)}{2}$, and we therefore relax a huge number of those constraints in problem P2. We include all constraints (4) for the relaxed sub-sequence π' though.

It is therefore clear that problem P2 is computationally more efficient than problem P1. This, however, has a drawback and that is the schedule obtained by problem P2 may not be feasible for the original problem P1. The reason is that we do not enforce the separation time requirement for all aircraft, except only for the adjacent aircraft in the nonrelaxed sub-sequence(s). For

example, for the presented instance with 100 aircraft, problem P1 introduces 4950 constraints (4), whereas problem P2 includes only 99 of those constraints (4). Due to the same reasoning, the objective function of problem P2 is a lower bound on the optimal objective function value of problem P1.

To implement problem P2 for the single-runway case, we use the following “or logical” (exclusive disjunction) constraints, which are features of CPLEX. Using the “or logical” constraints would lead to removal of the binary variables y_{ij} and constraints (5):

$$x_j \geq x_i + s_{ij} \quad \text{or} \quad x_i \geq x_j + s_{ji}, \quad j = i + 1 \quad \text{or} \quad i = j + 1. \quad (14)$$

We may follow the similar procedure and develop problem P2 for the multiple-runway case. This leads to the addition of constraints (15) and removal of constraints (4), (5), and (6) and binary variables y_{ij} :

$$x_j \geq x_i + s_{ij}(\gamma_{ir} + \gamma_{jr} - 1) \quad \text{or} \quad x_i \geq x_j + s_{ji}(\gamma_{ir} + \gamma_{jr} - 1), \quad j = i + 1 \quad \text{or} \\ i = j + 1, r = 1, \dots, m. \quad (15)$$

We note that constraints (15) do not include binary variables δ_{ij} . In summary, given a feasible sequence π , the problem P2 is formed by the objective function (1), constraints (2), (3), (7), (15), and variables γ_{ir} , x_i , α_i , β_i .

Next, we propose a procedure for dealing with the infeasible schedule generated by problem P2.

Feasibility of problem P2

Recall that problem P2 does not include all of constraints (4), meaning that the separation time requirement may not be applied between every pair of aircraft. Indeed, the separation time requirement is only applied between all aircraft in the relaxed sub-sequence and between the adjacent aircraft in the nonrelaxed sub-sequence(s). This may result in the schedule generated by problem P2 to be infeasible for problem P1, particularly if the separation times do not follow the triangular inequality.

The feasibility of the schedule produced by problem P2 can easily be verified, and then repaired if violated. Intuitively, if the landing schedule produced by problem P2 is given as the input to problem P1 (hence, all the decision variables in problem P1 turn into parameters), then solving problem P1 effectively results either in the same schedule as of problem P2, or in an infeasible status. The latter implies that some of the not-yet-added constraints (4) must be included in problem P2. Because we do not have *a priori* information on the number of those constraints and also to keep problem P2 efficiently solvable, we start by including constraints related to two immediate predecessors of an aircraft (initially we include one constraint (4) per aircraft, that is, the landing schedule of an aircraft only depends on its immediate predecessor, which is shown in Fig. 4a). To this end, we update problem P2 by adding an additional constraint per aircraft, which may be generated in a similar way to that of constraints (14) or (15). This is illustrated in Fig. 4b. Again, we solve problem P2 and check the feasibility of the so obtained schedule against problem P1. In case of infeasibility, we consider three preceding aircraft (Fig. 4c) for every aircraft, and follow the above procedure. No more than three preceding aircraft will be considered because there is no guarantee on the minimum number of preceding aircraft to be considered for each aircraft, in order to ensure that a

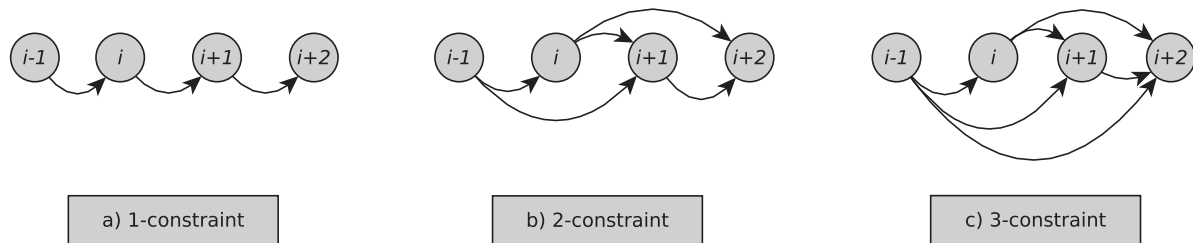


Fig. 4. Generating a feasible schedule for the ALP by problem P2: (a) the case of using one constraint (4) per aircraft in problem P2 (the default case), (b) the case of using two constraints (4) per aircraft, and (c) the case of using three constraints (4) per aircraft.

feasible schedule is delivered. Therefore, if no feasible schedule is produced by problem P2, problem P1 that contains all preceding constraints is solved, by considering time limits, to deliver a feasible schedule.

We use the solver CPLEX with a time limit for solving problem P2 because obtaining an optimal solution for problem P2 may still be a computational challenge. Setting reasonable time limits may not compromise the solution quality and can significantly improve the efficiency of solving the problem, which is important for real-world applications. We note that problem P2 reports an infeasible schedule only for one instance, out of the 49 tested instances in Section 4, which was then repaired by the above procedure. That indicates the effectiveness of problem P2 in generating feasible schedules for problem P1.

In order to further improve the efficiency of the proposed R&S matheuristic, in what follows we propose speed-up techniques that further expedite the solve procedure.

Speed-up procedures

Fast algorithms are very important for real-world applications of the ALP. In this section, we propose two speed-up procedures to further reduce the computation time of the solve procedure. The first speed-up is proposed for the single-runway case and the second one is designed for the multiple-runway case.

Recall that in the single-runway case the R&S algorithm starts from the beginning of a given sequence and progressively relaxes and solves a number of sub-sequences. Due to the penalties associated with early and late landings, changing the landing position of an aircraft too further forward or backward in the sequence is unlikely to be “profitable,” that is, it may not improve the objective function value. Therefore, we restrict the re-sequencing of an aircraft within a few positions backward and forward, which we refer to as the vicinity of the aircraft. The vicinity is controlled by the parameter $AR \in \mathbb{Z}^+$ (“adjacency radius”). This idea is graphically illustrated in Fig. 5.

For the multiple runway case, we observe that by employing more runways the number of aircraft landings on each runway decreases, and therefore, the chance of scheduling aircraft landings on their target landing time increases. This results in a considerable number of aircraft to have an earliness or lateness penalty of zero. In most circumstances, relaxing such aircraft would not lead to a better schedule and it only increases the computational burden. Hence, before relaxing a

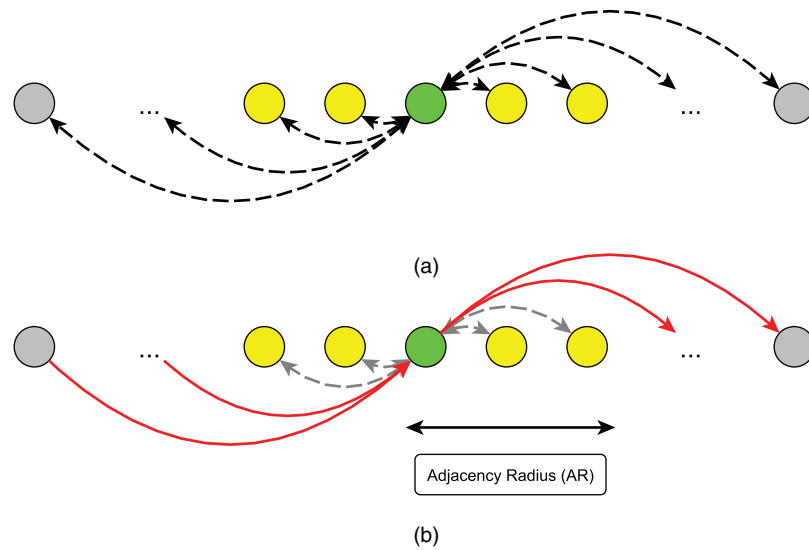


Fig. 5. The operation of speed-up procedure for the single-runway case: (a) a relaxed aircraft shown in green is connected either to neighboring aircraft (shown in yellow) that are located within a certain proximity, or to nonneighboring aircraft (shown in gray) by disjunctive arcs, and (b) due to parameter AR an aircraft is only re-sequenced with its neighbors that are within AR positions from the aircraft. The disjunctive dashed arcs highlight that and the conjunctive arcs are used to highlight the nonneighboring aircraft.

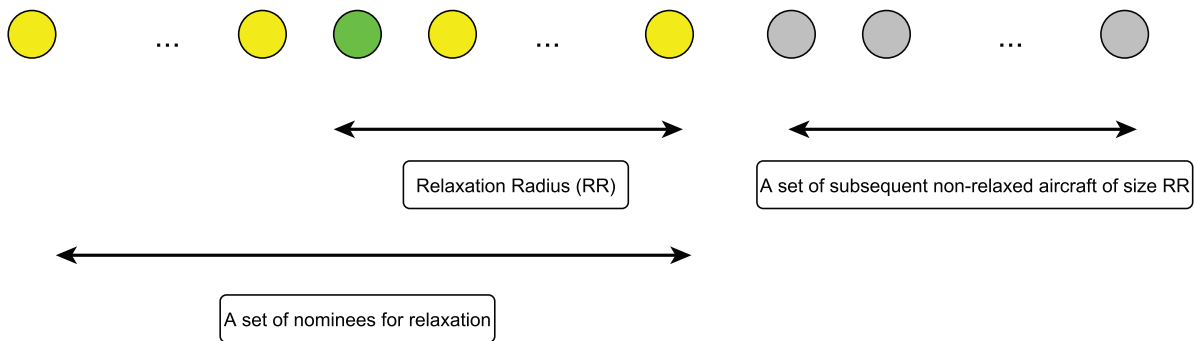


Fig. 6. A set of aircraft to be relaxed (shown in green and yellow), and a set of nonrelaxed aircraft (shown in gray), both of which include the aircraft in the relaxation radius (RR). If the landing penalties of all those aircraft are equal to 0, the solve procedure skips relaxing those aircraft and proceeds to the next sub-sequence.

sub-sequence the solve procedure preprocesses the landing penalties of the aircraft in the sub-sequence, as well as of the successor nonrelaxed aircraft (if the successor nonrelaxed aircraft are early or tardy delaying the relaxed aircraft may yield improved schedule) within the relaxation radius (RR); see Fig. 6. If the landing penalties of all those aircraft are equal to 0, the solve procedure skips relaxing the sub-sequence and considers the next sub-sequence.

We will discuss the impact of the speed-up procedures on the overall run time of the R&S algorithm in Section 4.2.

3.4. Updating the incumbent sequence

Upon obtaining a schedule with an improved objective function value by the R&S algorithm, the incumbent sequence π may need to be updated. This is easily performed by sorting the relaxed aircraft in nondecreasing order of their scheduled landing times (the value of decision variables x_i). It should be noted that because the order of nonrelaxed aircraft remains unchanged, the update scheme keeps their order as is.

3.5. Operation of the R&S algorithm

Following the detailed discussion of the components of the R&S algorithm in previous sections, Fig. 7 summarizes the implementation of the R&S algorithm for solving the ALP. The R&S first initializes the parameters. In the flowchart, π denotes the initial sequence generated by the ETLT construction heuristic (Algorithm 2 in Appendix A). The optimal schedule for π is generated through solving problem P1 by CPLEX. We denote by $z_1(\cdot)$ the objective function value of problem P1 and by $z_2(\cdot)$ that of problem P2, both given a sequence of aircraft landings. The R&S algorithm then iterates if and only if the initial sequence π leads to an objective function value greater than 0. At each iteration, a set of consecutive aircraft is relaxed from their landing position in the current sequence, and the aircraft are re-sequenced by solving problem P2 (or by problem P1 if problem P2 fails to produce a feasible landing schedule) and the current sequence is updated accordingly. That process continues until the stopping criterion is met. It should be noted that if the objective function value of problems P1 and P2 for the so obtained sequence do not match up and less than three immediate preceding aircraft were considered by problem P2 in the solve procedure (i.e., $f < 3$), the R&S is restarted by letting one more immediate preceding aircraft in problem P2, otherwise the obtained schedule is reported.

4. Computational results

We tested the proposed R&S algorithm on 13 standard benchmark instances of the ALP available at OR Library (<http://people.brunel.ac.uk/~mastjjb/jeb/orlib/airlandinfo.html>). The instances range from small problems (10 aircraft) to large ones (500 aircraft), and include up to five runways. Therefore, we studied a total of 49 instances, from which 13 instances use one runway and the remaining 36 instances utilize between two and five runways. We coded the R&S algorithm in the C++ programming language and we implemented problems P1 and P2 by using the CPLEX Concert Technology version 12.8.0 (IBM, 2017). Unless otherwise stated, we used default parameter settings for CPLEX. We performed the computational experiments on a personal computer with Intel® Core™ i5-6500 CPU clocked at 3.20GHz with 8GB of memory under Windows 10 operating system. The computing machine has four processors (threads). We utilized only one processor for CPLEX (both within the R&S and as the stand-alone).

Next, we explain the parameter tuning process followed by analyzing the effectiveness of the speed-up procedures in Section 4.2. We report the computational result of the R&S in Section 4.3 and we present the advantages of using the R&S algorithm over the routine of FCFS in Section 4.4.

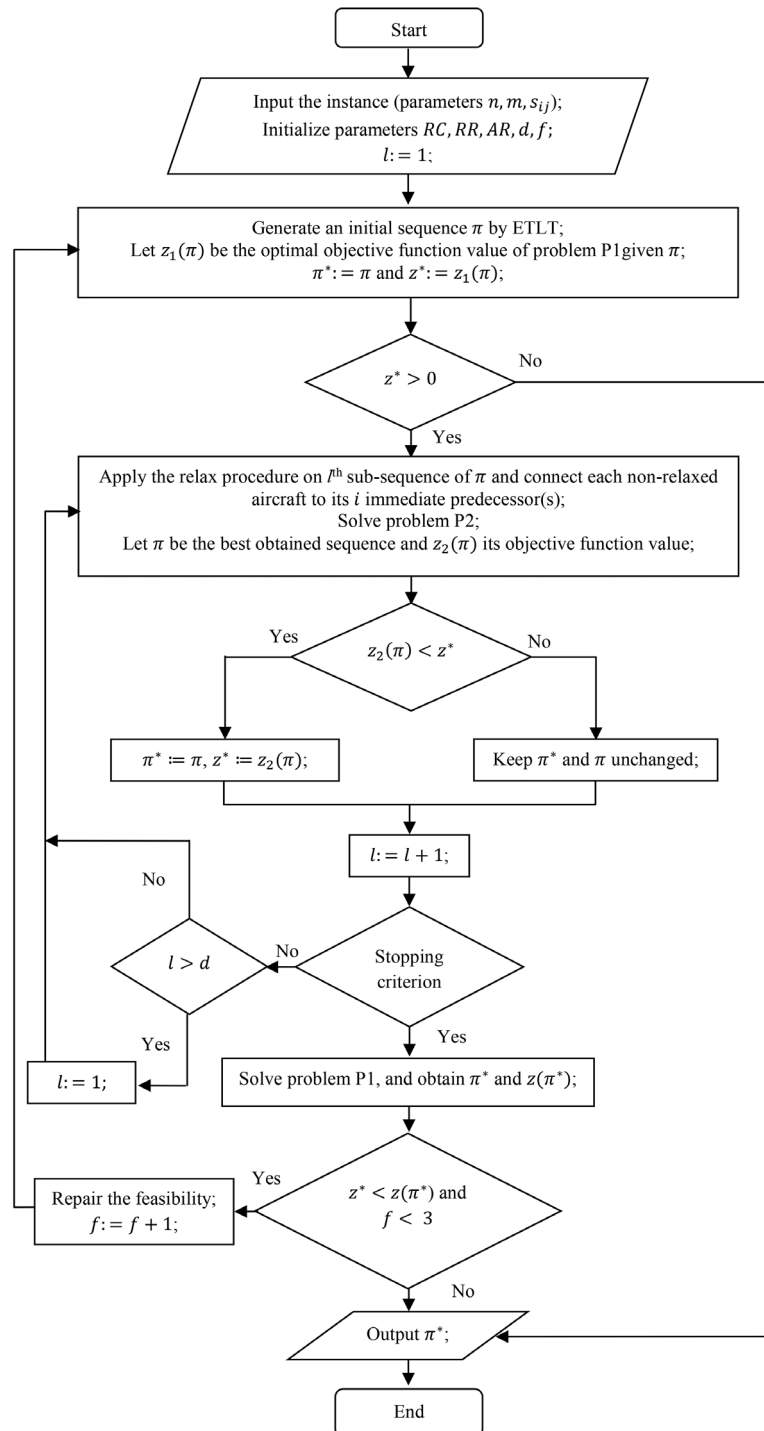


Fig. 7. Detailed operations of the R&S matheuristic algorithm for the ALP.

Table 2

Value of parameters for the R&S algorithm

Runway	No. of sub-sequences (d)		Relaxation radius (RR)		Adjacency radius (AR)		Time limit (seconds)	
	n	Value	n	Value	n	Value	n	Value
Single	$n \leq 20$	4	$n \leq 150$	$\max(4, \frac{n}{25})$	$n \leq 250$	5	$n \leq 100$	1
	$20 < n \leq 150$	25	$150 < n \leq 250$	$\max(4, \frac{n}{50})$	$n > 250$	6	$n > 100$	2
	$150 < n \leq 250$	50	$n > 250$	$\max(4, \frac{n}{150})$				
	$n > 250$	180						
Multiple	$n \leq 20$	4	–	6	–	–	$n \leq 250$	1
	$20 < n \leq 100$	25	–	6	–	–	$n > 250$	2
	$100 < n \leq 250$	40	–	6	–	–		
	$n > 250$	80	–	6	–	–		

4.1. Parameter tuning

For the purpose of parameter tuning, we do not run a full factorial design of experiments to choose the best value of the parameters because the combination of values of the parameters would result in a large number of experiments. Therefore, we conduct some experiments in which we solve a number of instances by using different values of parameters. We could not obtain a set of values for the parameters that work well on all instances. We, however, observe that the choice of values of the parameters depends on the size of the instances and further varies between the single runway and multiple runway cases. We therefore group the 13 instances with one runway into four classes of (1) small with up to 20 aircraft, medium that includes (2) between 20 and 150 aircraft and (3) between 150 and 250 aircraft, and (4) large with more than 250 aircraft. We also group the 36 instances with multiple runways into four classes of (1) small with up to 20 aircraft, medium in two different classes, (2) between 20 and 100 aircraft and (3) between 100 and 250 aircraft, and (4) large with more than 250 aircraft. Table 2 reports the values of the parameters, per each instance group, that we use to solve all instances of the group.

The reason for choosing those values of the parameters is because considering the impact of size of an instance, they result in good quality solutions in short times, which is in line with the aims of our study. For example, the smaller values of d (number of sub-sequences) typically lead to larger sub-sequences but fewer of them, which are more challenging to solve in short times; and we observe that while that does not contribute to the solution's quality, it greatly increases the computation time. Regarding parameters RR and AR , due to earliness and tardiness penalties we observe that it is not beneficial to schedule an aircraft far from its target landing time, neither earlier nor later, implying that larger values for those parameters may not be beneficial. We impose time limits for the solver CPLEX as an effective computation time reduction strategy. We choose the time limits such that good solutions are obtained in short computation times.

The R&S algorithm iteratively performs the relax and solve operations until one of the following three stopping criteria is met:

- Maximum number of iterations: For both single- and multiple-runway cases, we set that value to $\max(d, \min(\frac{n}{2}, 180))$.

Table 3

The impact of the speed-up procedures on the computation time (in seconds) of the R&S for large instances ($n > 100$)

Instance	Single runway		Multiple runways	
	Speed up	No speed-up	Speed-up	No speed-up
Airland10	36.97	54.15	5.95	8.52
Airland11	5.82	6.87	6.42	9.68
Airland12	24.07	32.44	5.52	10.33
Airland13	63.83	105.68	21.95	32.33

- Maximum number of iterations without improvement: We set the value of this parameter to 15 and 5 for the single and multiple runway cases. This criterion is only introduced when the number of executed iterations exceeds d (i.e., R&S has been through the whole sequence at least once).
- Objective function value of zero: The value of zero for the objective function means that an optimal schedule is delivered (though, not every optimal schedule has the value of zero, see Table B3 in Appendix B; we also note that the objective function cannot take a negative value).

4.2. Effectiveness of the speed-up procedures

As we discussed in Section 3.3, we apply two speed-up procedures in order to improve the computation time of the R&S algorithm. In this section, we show the effectiveness of those speed-up procedures by conducting two experiments on the four large instances with more than 100 aircraft. The first experiment is devoted to the speed-up for the single-runway case, and in the second experiment we analyze the speed-up for the multiple-runway case. We run the R&S algorithm five times with and without the speed-ups. We report the average computation times of the R&S (over five runs) in Table 3 for single and multiple runway cases. The table shows that the speed-up for the single runway case can reduce the computation time of solving the large instances between 15% and 40%. Also, the speed-up for the multiple-runway case improves the run time of the R&S between 30% and 45%. Those outcomes suggest the effectiveness of the speed-up procedures in reducing the run time of the proposed R&S algorithm.

4.3. Comparison across state-of-the-art algorithms

In this section, we compare the outcomes of the proposed R&S algorithm and those of CPLEX (for solving problem P1) and RH-VAR2-LS of Girish (2016) that we re-implemented. We chose RH-VAR2-LS because it was shown to be the best-performing algorithm in the literature for solving the ALP (Girish, 2016). We note that while we made a meticulous effort to re-implement the RH-VAR2-LS algorithm as close as possible to its original implementation reported in that study, our re-implementation may be slightly different from the operation of the original algorithm. This is because the original study by Girish (2016) did not fully disclose all the components of the RH-VAR2-LS algorithm.

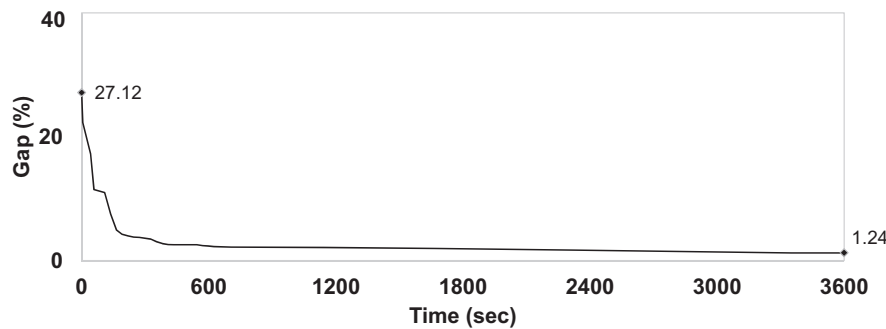


Fig. 8. CPLEX progress for instance Airland13 (500 aircraft) on one runway.

We initiate the CPLEX with the same initial landing sequence that we use for the R&S algorithm and we let the CPLEX run for a maximum of one hour. We choose the long run time of one hour for the CPLEX to show that the CPLEX will not benefit from long computation times. Also, we terminate the RH-VAR2-LS algorithm by following the stopping rules discussed in Girish (2016). That results in the computation times of RH-VAR2-LS to be significantly longer than the R&S, implying that RH-VAR2-LS benefits from longer run times. Despite these efforts, our re-implementation of the RH-VAR2-LS algorithm led to inferior solutions to those reported in the original study. For clarity, in Table 4 we report the outcomes of the re-implemented RH-VAR2-LS (column “RH-VAR2-LS (Girish, 2016)”), as well as the objective function values reported in Girish (2016) (column “BKS”).

In Table 4, the first three columns give the details for each instance including the name and the number of aircraft (n) and runways (m). The fourth column reports the best-known solution (BKS) for each instance, which is taken from Girish (2016). Columns five to seven show the objective values, optimality gaps upon termination, and the computation times for CPLEX. The remaining columns show the outcomes of RH-VAR2-LS of Girish (2016), which we re-implement on our machine and those of R&S, respectively, where the columns z^* , z_{avg} , and z_{std} show the best, average, and standard deviation of the objective function values obtained over five runs by each algorithm, and columns T_{avg} and T_{std} denote the average and standard deviation of the computation times of each algorithm over five runs.

According to Table 4, first, we observe that inferior outcomes of the re-implemented RH-VAR2-LS indicate that under the same computational settings our R&S is able to obtain superior schedules. Second, while the R&S algorithm outperforms both CPLEX and the re-implemented RH-VAR2-LS, it is the fastest method, indicating its suitability for practical settings. In particular, the R&S delivers the same or superior solutions (and never worse) to both CPLEX and RH-VAR2-LS, has the smallest run times that is bounded by around one minute, and has the standard deviation of 0 for all but one instance.

From Table 4, it is clear that the exact solvers such as CPLEX may not be a good choice for practical settings because of their long run times and poor quality of the generated schedules. For example, Fig. 8 shows the CPLEX performance in solving instance Airland13 (500 aircraft) with one runway (by optimizing problem P1), where the best schedule delivered by CPLEX after one hour of running is still 1.24% from the one produced by R&S in nearly one minute. Also, the CPLEX did not find that schedule within the first 40 minutes of running.

Table 4
The detailed outcomes of CPLEX, RH-VAR2-LS (Girish, 2016), and the R&S algorithm

Instance	n	m	BKS	CPLEX			RH-VAR2-LS (Girish, 2016)				R&S					
				z*	Gap (%)	T	z*	z _{avg}	z _{std}	T _{avg}	T _{std}	z*	z _{avg}	z _{std}	T _{avg}	T _{std}
Airland1	10	1	700	700	0	0.03	700	700	0	0.30	0.09	700	700	0	0.14	0.03
	2	90	90	90	0	0.03	90	90	0	0.27	0.26	90	90	0	0.29	0.05
	3	0	0	0	0	0.02	0	0	0	0.04	0.02	0	0	0	0.00	0.00
Airland2	15	1	1480	1480	0	0.08	1480	1492	10.95	0.42	0.16	1480	1480	0	0.21	0.02
	2	210	210	210	0	0.06	210	210	0	0.45	0.29	210	210	0	0.47	0.01
	3	0	0	0	0	0.02	0	0	0	0.05	0.03	0	0	0	1.08	0.44
Airland3	20	1	820	820	0	0.03	820	1108	280.57	0.37	0.20	820	820	0	0.14	0.00
	2	60	60	60	0	0.08	60	60	0	1.31	0.73	60	60	0	1.25	0.01
	3	0	0	0	0	0.03	0	0	0	0.09	0.03	0	0	0	0.01	0.00
Airland4	20	1	2520	2520	0	1.87	2520	2520	0	0.43	0.22	2520	2520	0	0.33	0.01
	2	640	640	640	0	8.94	640	640	0	1.31	0.67	640	640	0	2.77	0.07
	3	130	130	130	0	0.69	130	130	0	1.19	0.63	130	130	0	2.20	0.05
	4	0	0	0	0	0.05	0	0	0	0.08	0.04	0	0	0	0.35	0.01
Airland5	20	1	3100	3100	0	9.55	3680	4058	306.63	0.97	0.34	3100	3100	0	0.91	0.01
	2	650	650	650	0	8.94	650	680	22.36	1.30	0.69	650	650	0	3.63	0.02
	3	170	170	170	0	0.97	170	170	0	0.97	0.37	170	170	0	2.99	0.01
	4	0	0	0	0	0.05	0	0	0	0.08	0.04	0	0	0	0.33	0.00
Airland6	30	1	24,442	24,442	0	0	24,442	24,442	0	2.33	1.39	24,442	24,442	0	0.04	0.00
	2	554	554	554	0	0.14	568	614.2	36.72	4.27	2.24	554	554	0	0.62	0.01
	3	0	0	0	0	0.03	0	0	0	0.11	0.06	0	0	0	0.03	0.00
Airland7	44	1	1550	1550	0	0.16	1550	1550	0	4.76	2.46	1550	1550	0	0.17	0.01
	2	0	0	0	0	0.05	0	0	0	0.45	0.25	0	0	0	0.01	0.00
Continued																

Continued

Table 4
Continued

Instance	n	m	CPLEX			RH-VAR2-LS (Girish, 2016)						R&S			
			z^*	Gap (%)	T	z^*	z_{avg}	z_{std}	T_{avg}	T_{std}	z^*	z_{avg}	z_{std}	T_{avg}	T_{std}
Airland8	50	1	1950	0	0.19	2020	2206	181.19	2.01	1.17	1950	1950	0	1.48	0.02
		2	135	0	0.66	135	138	6.71	6.67	3.56	135	135	0	6.88	0.03
		3	0	0	0.2	0	0	0	0.33	0.17	0	0	0	0.02	0.00
Airland9	100	1	5611.7	25.23%	3600	5871.12	6200.57	478.71	10.58	5.63	5611.7	5611.7	0	3.79	0.06
		2	444.1	6.33%	3600	444.1	456.116	23.23	34.59	18.34	444.1	444.1	0	4.16	0.07
		3	75.75	0	0.83	75.75	221.818	168.25	17.41	12.25	75.75	75.75	0	2.39	0.01
		4	0	0	0.37	0	100.642	203.84	8.60	3.70	0	0	0	0.11	0.00
Airland10	150	1	12,292.2	52.04%	3600	12,616	13,311.1	620.43	29.18	15.74	12,292.2	12,293	1.79	36.97	0.57
		2	1143.7	79.26%	3600	1143.7	1143.81	0.10	63.15	33.30	1143.7	1143.7	0	8.43	0.03
		3	205.21	0	7.86	205.21	227.186	49.14	50.13	26.29	205.21	205.21	0	10.67	0.03
		4	34.22	0	2.22	34.22	166.47	183.40	22.84	14.34	34.22	34.22	0	3.88	0.01
		5	0	0	0.87	7.08	198.862	223.25	23.27	13.81	0	0	0	0.82	0.01
Airland11	200	1	12,418.32	37.74%	3600	12,682.2	13,190.4	303.19	30.80	16.40	12,418.32	12,418.32	0	5.82	0.02
		2	1330.91	87.15%	3600	1330.91	1355.67	27.03	91.29	47.91	1330.91	1330.91	0	10.04	0.05
		3	253.07	0	11.89	253.07	264.198	24.52	69.46	35.42	253.07	253.07	0	9.15	0.03
		4	54.53	0	3.93	54.53	182.27	181.46	57.56	32.66	54.53	54.53	0	5.03	0.03
		5	0	0	1.61	44.41	114.074	114.86	35.31	14.97	0	0	0	1.48	0.01
Airland12	250	1	16,122.18	42.84%	3600	16,466.8	17,079.5	397.87	55.73	30.35	16,122.18	16,122.18	0	24.07	0.01
		2	1695.62	90.87%	3600	1707.04	2202.068	1069.76	52.93	20.37	1695.62	1695.62	0	11.73	0.03
		3	221.97	0	22.23	221.97	289.164	148.54	68.04	36.34	221.97	221.97	0	5.71	0.01
		4	2.44	0	3.92	35.69	817.014	830.86	58.88	31.18	2.44	2.44	0	3.70	0.02
		5	0	0	2.37	57.04	142.02	71.01	48.93	29.17	0	0	0	0.93	0.03
Airland13	500	1	37,064.11	50.32%	3600	39,361.2	40,177.6	925.62	302.69	152.76	37,077.4	37,077.4	0	63.83	0.24
		2	3920.39	97.65%	3600	3931.33	4001.452	97.12	138.03	4.45	3920.39	3920.39	0	41.04	0.03
		3	673.85	90.02%	3600	691.85	864.074	216.17	239.73	136.35	673.85	673.85	0	30.18	0.04
		4	89.95	0	16.86	89.95	179.634	120.05	201.63	98.77	89.95	89.95	0	12.72	0.02
		5	0	0	11.29	193.6	340.074	126.77	143.72	85.41	0	0	0	3.86	0.03

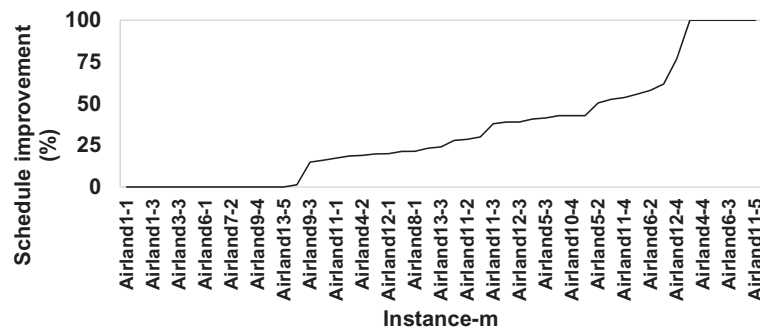


Fig. 9. Schedule improvement obtained by using the R&S algorithm compared to sequencing the aircraft by using the FCFS dispatching rule. The graph is obtained by sorting the values of improvement in nondecreasing order.

In conclusion, the practitioner may choose the proposed R&S algorithm for the following reasons. First, as Table 4 details, the R&S is the top performing method with respect to both the quality of the generated schedules and the run times. Second, the implementation of R&S is simple and straightforward as the R&S only relies on a generic decomposition-based framework. Third, the R&S uses the standard solver CPLEX, which has a demonstrated and established performance. The latter is important because it provides the practitioner with the extensive access to the support of CPLEX, and removes the need for developing customized heuristic and meta-heuristic methods that require extensive research and without guaranteeing the outcomes due to the heuristic nature of the methods. In our opinion, the conceptual simplicity of R&S is a significant advantage because it makes the R&S algorithm to be easily extended to other challenging combinatorial optimization problems.

4.4. Improvement over the FCFS

It is a well-known practice that the air traffic controllers order the aircraft by using the FCFS dispatching rule. An FCFS sequence can be obtained by sorting the aircraft in nondecreasing order of their target landing times (arrival times to the airport) and allocating them to the available runways, as illustrated by the ETLT (see Algorithm 2 in Appendix A). An optimal schedule for the FCFS sequence can be generated by optimizing problem P1 given the FCFS sequence. We observe that over all tested benchmark instances the R&S algorithm improves an FCFS schedule by up to 100%. This is further acknowledged by Fig. 9, which shows the amount of schedule improvement (in %) attained by the R&S algorithm over the FCFS schedule. The percent of improvement was calculated as

$$\frac{z_{fcfs} - z}{z_{fcfs}} \times 100,$$

where z_{fcfs} is the objective function value of the FCFS sequence and z is that of the R&S. It should be noted that even though the benchmark instances that we investigate in this paper have

been randomly generated, significant improvements over the FCFS are expected to be achievable in practice if an optimization technique, including the R&S algorithm, is used.

5. Conclusion

We proposed an efficient R&S matheuristic algorithm for solving the ALP. The core idea behind the algorithm is iterative deconstruction and reconstruction of a sub-sequence of aircraft landings. By solving 49 benchmark instances and comparing the outcomes and the state-of-the-art algorithm and the solver CPLEX, we showed that the proposed R&S algorithm obtains the best-known solutions and is quick enough to be implemented in real time. Those advantages of the R&S algorithm along with its algorithmic simplicity and ease of implementation contribute to the applicability of the proposed R&S for real-world applications. This is very important because due to the typical short time window available for planning the aircraft landings, delivering high-quality landing schedules or updating the available schedules in a short time is very important, and therefore, fast and effective algorithms are paramount.

Although we utilized the solver CPLEX in the solve procedure of the R&S algorithm, mainly due to the high performance of CPLEX, that does not limit the usability and applicability of the R&S algorithm because other exact methods including the branch-and-bound can be used instead. As future research directions, one may extend the R&S algorithm for the case of separation time requirements between pairs of aircraft landing on different runways. Another direction can be generalizing the R&S for the variants of nonidentical and dedicated runways. Also, investigations may be performed around adaptability of the R&S to other challenging combinatorial optimization problems.

Acknowledgments

We would like to thank the anonymous reviewers for their constructive comments on the earlier version of the paper. Mohammad Mahdi Ahmadian is the recipient of UTS International Research Scholarship (IRS) and UTS Faculty of Science Scholarship. Amir Salehipour is the recipient of an Australian Research Council Discovery Early Career Researcher Award (project number DE170100234) funded by the Australian Government.

References

- Abela, J., Abramson, D., Krishnamoorthy, M., De Silva, A., Mills, G., 1993. Computing optimal schedules for landing aircraft. *Proceedings of the 12th National Conference of the Australian Society for Operations Research*, Adelaide, pp. 71–90.
- Ahmadian, M.M., Salehipour, A., Cheng, T., 2020. A meta-heuristic to solve the just-in-time job-shop scheduling problem. *European Journal of Operational Research* 288, 1, 14–29.
- Awasthi, A., Kramer, O., Lässig, J., 2013. Aircraft landing problem: an efficient algorithm for a given landing sequence. 2013 IEEE 16th International Conference on Computational Science and Engineering, pp. 20–27.

- Balakrishnan, H., Chandran, B.G., 2010. Algorithms for scheduling runway operations under constrained position shifting. *Operations Research* 58, 6, 1650–1665.
- Beasley, J.E., Krishnamoorthy, M., Sharaiha, Y.M., Abramson, D., 2000. Scheduling aircraft landings—the static case. *Transportation Science* 34, 2, 180–197.
- Beasley, J.E., Krishnamoorthy, M., Sharaiha, Y.M., Abramson, D., 2004. Displacement problem and dynamically scheduling aircraft landings. *Journal of the Operational Research Society* 55, 1, 54–64.
- Bennell, J.A., Mesgarpour, M., Potts, C.N., 2011. Airport runway scheduling. *4OR* 9, 2, 115.
- Bennell, J.A., Mesgarpour, M., Potts, C.N., 2017. Dynamic scheduling of aircraft landings. *European Journal of Operational Research* 258, 1, 315–327.
- Bianco, L., Dell’Olmo, P., Giordani, S., 1999. Minimizing total completion time subject to release dates and sequence dependent processing times. *Annals of Operations Research* 86, 393–415.
- Boschetti, M.A., Maniezzo, V., Roffilli, M., Bolufé Röhrer, A., 2009. Matheuristics: optimization, simulation and control. In Blesa, M.J., Blum, C., Di Gasparo, L., Roli, A., Sampels, M., Schaerf, A. (eds) *Hybrid Metaheuristics*. Springer, Berlin–Heidelberg, pp. 171–177.
- D’Ariano, A., Pacciarelli, D., Pistelli, M., Pranzo, M., 2015. Real-time scheduling of aircraft arrivals and departures in a terminal maneuvering area. *Networks* 65, 3, 212–227.
- Doi, T., Nishi, T., Voß, S., 2018. Two-level decomposition-based matheuristic for airline crew rostering problems with fair working time. *European Journal of Operational Research* 267, 2, 428–438.
- Ernst, A.T., Krishnamoorthy, M., Storer, R.H., 1999. Heuristic and exact algorithms for scheduling aircraft landings. *Networks* 34, 3, 229–241.
- Faye, A., 2015. Solving the aircraft landing problem with time discretization approach. *European Journal of Operational Research* 242, 3, 1028–1038.
- Fuentes, M., Cadarso, L., Marín, Á., 2018. A fix & relax matheuristic for the crew scheduling problem. *Transportation Research Procedia* 33, 307–314.
- Furini, F., Kidd, M.P., Persiani, C.A., Toth, P., 2015. Improved rolling horizon approaches to the aircraft sequencing problem. *Journal of Scheduling* 18, 5, 435–447.
- Furini, F., Persiani, C.A., Toth, P., 2012. Aircraft sequencing problems via a rolling horizon algorithm. In Mahjoub, A.R., Markakis, V., Milis, I., Paschos, V.T. (eds) *International Symposium on Combinatorial Optimization*. Springer, Berlin, pp. 273–284.
- Ghoniem, A., Farhadi, F., 2015. A column generation approach for aircraft sequencing problems: a computational study. *Journal of the Operational Research Society* 66, 10, 1717–1729.
- Ghoniem, A., Farhadi, F., Reihaneh, M., 2015. An accelerated branch-and-price algorithm for multiple-runway aircraft sequencing problems. *European Journal of Operational Research* 246, 1, 34–43.
- Girish, B., 2016. An efficient hybrid particle swarm optimization algorithm in a rolling horizon framework for the aircraft landing problem. *Applied Soft Computing* 44, 200–221.
- Hansen, J.V., 2004. Genetic search methods in air traffic control. *Computers & Operations Research* 31, 3, 445–459.
- Helber, S., Sahling, F., 2010. A fix-and-optimize approach for the multi-level capacitated lot sizing problem. *International Journal of Production Economics* 123, 2, 247–256.
- Hu, X.B., Chen, W.H., 2005. Genetic algorithm based on receding horizon control for arrival sequencing and scheduling. *Engineering Applications of Artificial Intelligence* 18, 5, 633–642.
- IBM, 2017. *IBM ILOG CPLEX V12.8.0: User’s manual for CPLEX*. IBM, Armonk, NY.
- Maniezzo, V., Stützle, T., Voß, S., 2009. *Matheuristics: Hybridizing Metaheuristics and Mathematical Programming*. Springer, Berlin.
- Meng, X., Zhang, P., Li, C., 2011. Sliding window algorithm for aircraft landing problem. 2011 Chinese Control and Decision Conference (CCDC), pp. 874–879.
- Moser, I., Hendtlass, T., 2007. Solving dynamic single-runway aircraft landing problems with extremal optimisation. SCIS’07 IEEE Symposium on Computational Intelligence in Scheduling, pp. 206–211.
- Ng, K., Lee, C., Chan, F.T., Lv, Y., 2018. Review on meta-heuristics approaches for airside operation research. *Applied Soft Computing* 66, 104–133.
- Ng, K., Lee, C., Chan, F.T., Qin, Y., 2017. Robust aircraft sequencing and scheduling problem with arrival/departure delay using the min-max regret approach. *Transportation Research Part E: Logistics and Transportation Review* 106, 115–136.

- Pinol, H., Beasley, J., 2006. Scatter search and bionomic algorithms for the aircraft landing problem. *European Journal of Operational Research* 171, 2, 439–462.
- Sabar, N.R., Kendall, G., 2015. An iterated local search with multiple perturbation operators and time varying perturbation strength for the aircraft landing problem. *Omega* 56, 88–98.
- Salehipour, A., 2020. An algorithm for single- and multiple-runway aircraft landing problem. *Mathematics and Computers in Simulation* 175, 179–191.
- Salehipour, A., Ahmadian, M.M., 2017. A heuristic algorithm for the aircraft landing problem. The 22nd International Congress on Modelling and Simulation (MODSIM2017).
- Salehipour, A., Ahmadian, M.M., Oron, D., 2018. Efficient and simple heuristics for the aircraft landing problem. Matheuristic 2018 Workshop (MH2018).
- Salehipour, A., Modarres, M., Naeni, L.M., 2013. An efficient hybrid meta-heuristic for aircraft landing problem. *Computers & Operations Research* 40, 1, 207–213.
- Salehipour, A., Naeni, L.M., Kazemipoor, H., 2009. Scheduling aircraft landings by applying a variable neighborhood descent algorithm: runway-dependent landing time case. *Journal of Applied Operational Research* 1, 1, 39–49.
- Samá, M., D'Ariano, A., Corman, F., Pacciarelli, D., 2018. Coordination of scheduling decisions in the management of airport airspace and taxiway operations. *Transportation Research Part A: Policy and Practice* 114, 398–411.
- Samá, M., D'Ariano, A., D'Ariano, P., Pacciarelli, D., 2017. Scheduling models for optimal aircraft traffic control at busy airports: tardiness, priorities, equity and violations considerations. *Omega* 67, 81–98.
- Vadlamani, S., Hosseini, S., 2014. A novel heuristic approach for solving aircraft landing problem with single runway. *Journal of Air Transport Management* 40, 144–148.
- Wen, M., Larsen, J., Clausen, J., 2005. An exact algorithm for Aircraft Landing Problem. Technical report, Informatics and Mathematical Modelling, Technical University of Denmark, DTU.
- Woo, Y.B., Kim, B.S., 2018. Matheuristic approaches for parallel machine scheduling problem with time-dependent deterioration and multiple rate-modifying activities. *Computers & Operations Research* 95, 97–112.
- Yu, S.P., Cao, X.B., Zhang, J., 2011. A real-time schedule method for aircraft landing scheduling problem based on cellular automation. *Applied Soft Computing* 11, 4, 3485–3493.
- Zhan, Z.h., Zhang, J., Gong, Y.j., 2009. Ant colony system based on receding horizon control for aircraft arrival sequencing and scheduling. Proceedings of the 11th Annual Conference on Genetic and Evolutionary Computation, pp. 1765–1766.
- Zhang, D., Klabjan, D., 2017. Optimization for gate re-assignment. *Transportation Research Part B: Methodological* 95, 260–284.

Appendix A: The ETLT algorithm

The ETLT algorithm (Salehipour et al., 2013) is summarized in Algorithm 2.

Algorithm 2. The earliest target landing time (ETLT) construction heuristic

Input: An instance of the ALP, where I and R are sets of aircraft and runways, $\omega = \{\}$ and $\pi_{2 \times n} = \{\}$.

Output: A feasible landing sequence π for the ALP.

Initialization:

$r := 1$ (selecting the first runway);

$k := 1$;

Let $\omega := (\sigma(1), \sigma(2), \dots, \sigma(n))$, where $T_{\sigma(1)} \leq T_{\sigma(2)} \leq \dots \leq T_{\sigma(n)}$, be the sorted sequence of aircraft landings (sorted in non-decreasing order of target landing times);

$\pi[1][k] := \omega[1]$;

// The first aircraft (i.e., $\omega[1]$) appears first in π

$\pi[2][k] := r$;

// Assign the first aircraft (i.e., $\omega[1]$) to runway 1

Remove the first element from ω ;

Allocation:

```

while  $\omega \neq \{\}$  do
   $k := k + 1$ ;
  if  $|\omega| = 1$  then
     $\pi[1][k] := \omega[1]$  and  $\pi[2][k] := r$ ;
    Remove the first element from  $\omega$ ;
  else
    if  $T_{\omega[2]} < T_{\omega[1]} + s_{\omega[1], \omega[2]}$  then
      // Assign the first two aircraft in  $\omega$  to two different runways
       $\pi[1][k] := \omega[1]$  and  $\pi[2][k] := r$ ;
       $k := k + 1$ ;
       $r := r + 1$ ;
      if  $r > m$  then
         $r := 1$ ;
      end
       $\pi[1][k] := \omega[2]$  and  $\pi[2][k] := r$ ;
      Remove the first two elements from  $\omega$ ;
    else
      // Assign the first two aircraft in  $\omega$  to the same runway
       $\pi[1][k] := \omega[1]$  and  $\pi[2][k] := r$ ;
       $k := k + 1$ ;
       $\pi[1][k] := \omega[2]$  and  $\pi[2][k] := r$ ;
      Remove the first two elements from  $\omega$ ;
    end
     $r := r + 1$ ;
    if  $r > m$  then
       $r := 1$ ;
    end
  end
end
return  $\pi$ ;

```

Appendix B: A numerical example

Consider the instance Airland2 that includes 15 aircraft. Table B1 details the landing data for the instance. The separation time between every pair of aircraft landing on the same runway is shown in Table B2. Table B3 presents the landing sequences generated by the ETLT (Algorithm 2) on one and two runways and their schedule delivered by the CPLEX.

Table B1
The landing data for Airland2

Aircraft	E_i	T_i	L_i	c_i^-	c_i^+
1	129	155	559	10	10
2	190	250	732	10	10
3	84	93	501	30	30
4	89	98	509	30	30
5	100	111	536	30	30
6	107	120	552	30	30
7	109	121	550	30	30
8	109	120	544	30	30
9	115	128	557	30	30
10	134	151	610	30	30
11	266	341	837	10	10
12	251	313	778	10	10
13	160	181	674	30	30
14	152	171	637	30	30
15	276	342	815	10	10

Table B2
The separation times between pairs of aircraft for Airland2

(i, j)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	–	3	15	15	15	15	15	15	15	15	3	3	15	15	3
2	3	–	15	15	15	15	15	15	15	15	3	3	15	15	3
3	15	15	–	8	8	8	8	8	8	8	15	15	8	8	15
4	15	15	8	–	8	8	8	8	8	8	15	15	8	8	15
5	15	15	8	8	–	8	8	8	8	8	15	15	8	8	15
6	15	15	8	8	8	–	8	8	8	8	15	15	8	8	15
7	15	15	8	8	8	8	–	8	8	8	15	15	8	8	15
8	15	15	8	8	8	8	8	–	8	8	15	15	8	8	15
9	15	15	8	8	8	8	8	8	–	8	15	15	8	8	15
10	15	15	8	8	8	8	8	8	8	–	15	15	8	8	15
11	3	3	15	15	15	15	15	15	15	15	–	3	15	15	3
12	3	3	15	15	15	15	15	15	15	15	3	–	15	15	3
13	15	15	8	8	8	8	8	8	8	8	15	15	–	8	15
14	15	15	8	8	8	8	8	8	8	8	15	15	8	–	15
15	3	3	15	15	15	15	15	15	15	15	3	3	15	15	–

Table B3

The sequences generated by Algorithm 2 and their associated schedule delivered by CPLEX for Airland2

$m = 1$	i	3	4	5	6	8	7	9	10	1	14	13	2	12	11	15	$z = 1500$
	r	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
	x_i	88	96	104	112	120	128	136	144	159	174	182	250	313	341	344	
	$c_i^- \alpha_i + c_i^+ \beta_i$	150	60	210	240	0	210	240	210	40	90	30	0	0	0	20	
$m = 2$	i	3	4	5	6	8	7	9	10	1	14	13	2	12	11	15	$z = 300$
	r	1	1	1	2	1	2	1	2	1	2	2	1	1	2	1	
	x_i	90	98	111	113	120	121	128	151	155	171	181	250	313	341	342	
	$c_i^- \alpha_i + c_i^+ \beta_i$	90	0	0	210	0	0	0	0	0	0	0	0	0	0	0	